

DSA02- COMPUTER VISION WITH OPENCV

ASSIGNMENT

Name: Chalamala Sai Harshitha

Reg No. 192424346

A. Assignment Information

Department:	(AI & DS)
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	DSA0204- COMPUTER VISION WITH OPENCV
Academic Year / Batch	2026-2027
Faculty Name	Dr. G. A. SENTHIL
Assignment Title	Analyse and design a computer vision-based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.
Date of Issue	31-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) - CO	CO1: Implement fundamental image processing techniques required for computer vision and understand the image formation process. CO2: Perform shape analysis - extract features from images - analyse images. CO3: Generate 3D models from images. CO4: Develop applications using computer vision techniques. CO5: Understand video processing - motion computation - 3D vision - geometry.

Bloom's Taxonomy Level	. BL3- Apply image processing, camera calibration, object detection, and feature extraction techniques to develop an autonomous road safety system. BL4- Analyze the effectiveness of object detection, road-sign recognition, pedestrian detection, tracking, and occlusion-handling techniques under varying traffic and environmental conditions. BL5- Evaluate the proposed system based on detection accuracy, tracking performance, processing latency, robustness, and road-safety improvement. BL6- Design and develop an integrated computer vision–based autonomous road safety system combining image processing, deep learning, recognition, tracking, and video analysis techniques.
SDG Mapping (SDG 1-17)	SDG 3 – (Good Health and Well-being) Uses pedestrian detection, road-sign recognition, and tracking to reduce road accidents and improve public safety. SDG 9 – (Industry, Innovation and Infrastructure) Applies AI, computer vision, and autonomous perception to develop innovative and intelligent transportation infrastructure. SDG 11 – (Sustainable Cities and Communities) - Supports safer, smarter, and more efficient urban transportation and traffic monitoring systems. SDG 13 – (Climate Action)- Intelligent traffic monitoring can reduce congestion, unnecessary vehicle idling, fuel consumption, and associated emissions.
Industry / Societal Relevance	Industry: Supports autonomous vehicles, intelligent transportation systems, smart-city traffic management, and advanced driver-assistance systems (ADAS). Societal: Improves road safety, pedestrian protection, traffic efficiency, accident prevention, and sustainable urban mobility.

Assignment Question

Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Road accidents are frequently caused by factors such as poor visibility, failure to detect pedestrians and vehicles, missed road signs and markings, traffic congestion, and temporary occlusions. An autonomous road-safety system can address these challenges by continuously analysing camera images and video using computer vision techniques.

The proposed system is a computer vision–based autonomous road safety system that integrates camera calibration, object detection, pedestrian detection, road-sign recognition, road-marking recognition, multi-object tracking, and occlusion handling. The system processes road-scene images or video captured by a camera and identifies important objects and road features in real time. It tracks moving objects across successive video frames and handles situations where objects are partially or temporarily hidden.

The system is further analysed under different traffic and environmental conditions, such as normal traffic, heavy traffic, low-light conditions, and partial occlusion.

1.2 Problem Formulation

The problem can be formulated as follows:

Input:

Camera-captured road images or video containing vehicles, pedestrians, road signs, road markings, and other traffic elements under varying traffic and environmental conditions.

Processing:

The system performs:

1. Camera calibration and image preprocessing.
2. Vehicle and object detection.
3. Pedestrian detection.
4. Road-sign recognition.
5. Road-marking recognition.
6. Multi-object tracking across video frames.
7. Occlusion handling for partially or temporarily hidden objects.
8. Performance analysis under different traffic and environmental conditions.

Output:

The system produces detected and recognised road objects/features, their locations and tracking information, together with safety-related information that can support autonomous road-safety decisions.

1.3 Performance Evaluation

The proposed system is evaluated using the following measures:

- **Detection Accuracy** – how accurately vehicles, pedestrians, signs, and other relevant objects are detected.
- **Tracking Performance** – how effectively objects are tracked across consecutive video frames.
- **Processing Latency** – the time required to process each frame or video sequence.
- **Robustness** – the ability of the system to operate under varying traffic and environmental conditions.
- **Road-Safety Improvement** – the potential of the system to improve pedestrian protection, accident prevention, and overall road safety.

This formulation directly addresses the assignment requirement to integrate the specified computer-vision components and evaluate their performance in realistic conditions.

2. Objective and Expected Outcomes

Objectives

The main objective of the proposed system is to develop a computer vision–based autonomous road safety system that can analyse road scenes and identify vehicles, pedestrians, road signs, and road markings. The system aims to combine different computer vision techniques to provide reliable information about the road environment and support safer transportation.

- To design a computer vision–based autonomous road safety system for analysing road environments and identifying potential safety-related objects and features.
- To perform camera calibration to reduce image distortion and improve the reliability of the visual information obtained from the camera.
- To apply image preprocessing techniques such as noise reduction, grayscale conversion, and image enhancement to improve the quality of input images.
- To detect vehicles and pedestrians accurately from road-scene images and determine their locations using object detection techniques.
- To recognise road signs and identify important visual indicators that provide information about traffic rules and road conditions.
- To detect road markings and lane information to understand the structure of the road and support safer movement.

- To apply multi-object tracking for continuously monitoring vehicles and pedestrians across consecutive video frames.
- To handle partial and temporary occlusion so that objects can still be identified or tracked when they are blocked by other vehicles or objects.
- To analyse the performance of the system under different traffic and environmental conditions, including normal traffic, heavy traffic, low-light conditions, and partially occluded scenes.
- To evaluate the system using suitable performance measures such as detection accuracy, precision, recall, F1-score, tracking performance, detection confidence, and processing latency.
- To analyse the limitations and practical challenges of using computer vision for real-world road safety applications.
- To study the practical relevance of the proposed system in intelligent transportation systems, autonomous vehicles, traffic monitoring, accident prevention, pedestrian protection, and public safety.

Expected Outcomes

The proposed system is expected to produce a complete computer vision–based framework capable of analysing road-scene images and providing meaningful road-safety information.

- Accurate detection and localisation of vehicles and pedestrians present in the road scene.
- Identification of important road signs and road markings using suitable image processing and recognition techniques.
- Improved image quality and more reliable processing through camera calibration and preprocessing.
- Continuous monitoring of multiple road objects through multi-object tracking when video input is available.
- Better handling of objects that are partially or temporarily occluded by other vehicles or objects.
- Generation of detection confidence values that indicate the reliability of individual object detections.
- Measurement of processing latency to determine the time required for analysing the input image or frame.

- Performance comparison under different traffic and environmental conditions to identify situations where the system performs effectively or faces difficulties.
- Evaluation of the overall system using quantitative and qualitative observations obtained from the experimental results.
- Identification of system limitations, such as reduced performance in poor lighting, crowded traffic, and severe occlusion.
- Demonstration of how computer vision can assist in road monitoring, pedestrian safety, accident prevention, and intelligent transportation.
- Development of a practical foundation that can be further extended from single-image analysis to real-time video-based autonomous road safety applications.

3. Requirements, Constraints and Assumptions

Requirements

- Road images or video captured using a camera.
- Python and OpenCV for image and video processing.
- Techniques for detecting vehicles and pedestrians.
- Methods for recognising road signs and road markings.
- Multi-object tracking for tracking vehicles and pedestrians.
- Occlusion handling for partially hidden objects.
- Performance evaluation under different traffic and environmental conditions.

Category	Requirement
Processor	Intel/AMD processor or equivalent
RAM	4 GB or above
Camera	Webcam or road-scene video
Operating System	Windows / Linux
Programming Language	Python
Libraries	OpenCV, NumPy, Matplotlib
Development Platform	Google Colab / Jupyter Notebook

Constraints

- Poor lighting and weather conditions may reduce detection accuracy.
- Heavy traffic can make object detection and tracking difficult.
- Occlusion may cause temporary loss of detected objects.

- High-quality processing may require better computational resources.
- Camera position and image quality can affect system performance.

Assumptions

- The input images or videos are clear enough for analysis.
- The camera is properly positioned and calibrated.
- Vehicles, pedestrians, signs, and road markings are present in the input data.
- The system is tested under different traffic and environmental conditions.
- The proposed system is considered a prototype for road-safety analysis.

4. Application of Relevant Course Knowledge / Concepts

The proposed road-safety system applies the following computer vision concepts:

- **Image Processing:** Used for resizing, noise reduction, grayscale conversion, and enhancement of road images.
- **Camera Calibration:** Used to correct lens distortion and improve the accuracy of the road-scene analysis.
- **Object Detection:** Used to detect vehicles, pedestrians, and other road objects.
- **Feature Extraction:** Used to identify important visual features of road signs and markings.
- **Image Segmentation:** Used to separate road regions and lane markings from the surrounding environment.
- **Shape Analysis:** Used for recognising road signs based on their shape and appearance.
- **Video Processing:** Used to analyse continuous frames from road-scene videos.
- **Multi-Object Tracking:** Used to maintain the identity and movement of multiple vehicles and pedestrians.
- **Occlusion Handling:** Used to continue tracking objects when they are temporarily blocked.

Performance Measures and Formulas

The system can be evaluated using standard performance metrics:

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1-Score:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Processing Latency:

$$Latency = T_{output} - T_{input}$$

where **TP** represents correctly detected objects, **TN** represents correctly identified non-objects, **FP** represents incorrect detections, and **FN** represents missed detections.

These concepts are relevant to the course outcomes covering image processing, feature and shape analysis, computer vision applications, and video processing.

5. Design / Proposed Solution / Methodology

5.1 Proposed Solution

The proposed system is an integrated **computer vision–based autonomous road safety system** that processes road images or video to detect, recognise, and track important road elements.

5.2 Methodology

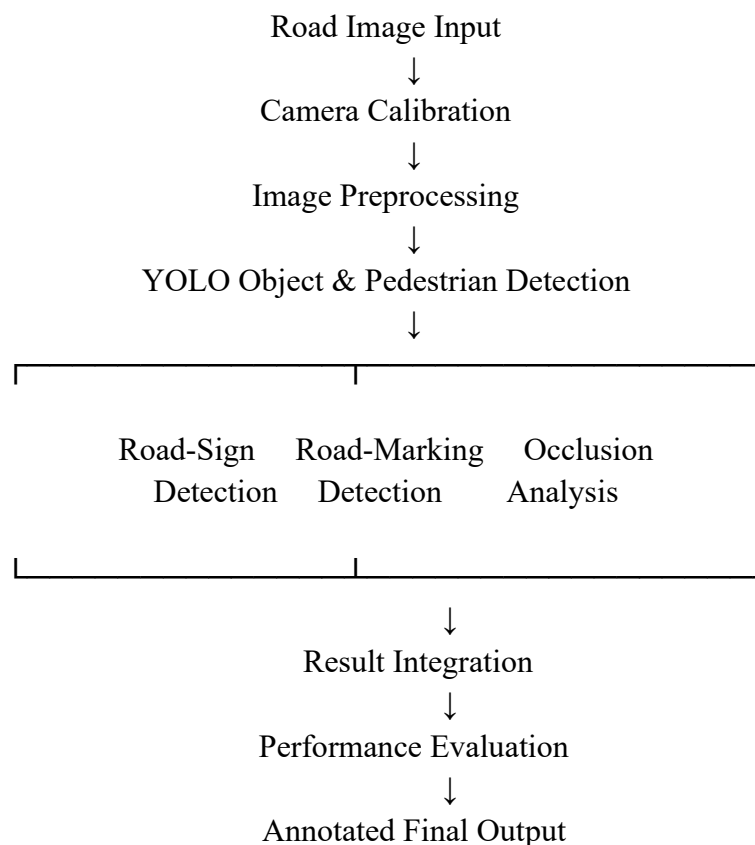
The system follows these main steps:

1. **Input Acquisition** – Capture road-scene images or video using a camera.
2. **Camera Calibration** – Calibrate the camera and reduce lens distortion.
3. **Image Preprocessing** – Improve the quality of input frames for further analysis.
4. **Object and Pedestrian Detection** – Detect vehicles, pedestrians, and other relevant objects.
5. **Road-Sign Recognition** – Identify and classify traffic signs.
6. **Road-Marking Recognition** – Detect lane markings and other road markings.

7. **Multi-Object Tracking** – Track detected vehicles and pedestrians across consecutive frames.
8. **Occlusion Handling** – Maintain detection and tracking when objects are partially hidden.
9. **Performance Evaluation** – Evaluate detection accuracy, tracking performance, processing latency, and robustness under different traffic and environmental conditions.

Design

The proposed Autonomous Road Safety System is designed as a modular computer vision pipeline that processes a road-scene image and extracts important safety-related information. The design combines image processing, object detection, road-marking detection, road-sign candidate detection, and basic occlusion analysis.



The system begins by accepting a road image as input. The image is first passed through the camera calibration stage, where an estimated camera matrix is generated and distortion correction is applied. The calibrated image is then preprocessed using Gaussian filtering, grayscale conversion, and histogram equalisation to improve the quality of the image.

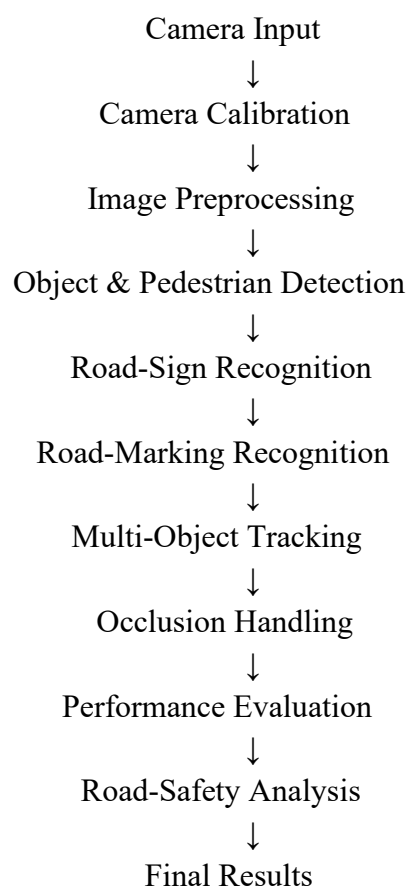
The processed image is given to the YOLO object detection model for detecting road-safety objects such as persons, cars, motorcycles, buses, trucks, and bicycles. Bounding boxes and confidence values are generated for the detected objects.

For road-marking detection, the system uses Canny edge detection, a region of interest, and the Hough Line Transform to identify lane-like road markings. Road-sign candidates are identified using HSV colour segmentation and contour analysis, particularly for red-coloured sign candidates.

The design also includes basic spatial occlusion analysis by calculating the Intersection over Union (IoU) between overlapping detection boxes. This helps identify possible cases where objects may be partially overlapping.

Finally, all detected information is combined into an annotated output image containing object bounding boxes, detection labels, road markings, road-sign candidates, and system information such as object, vehicle, pedestrian, and sign counts. The system also generates performance results including detection confidence and processing latency.

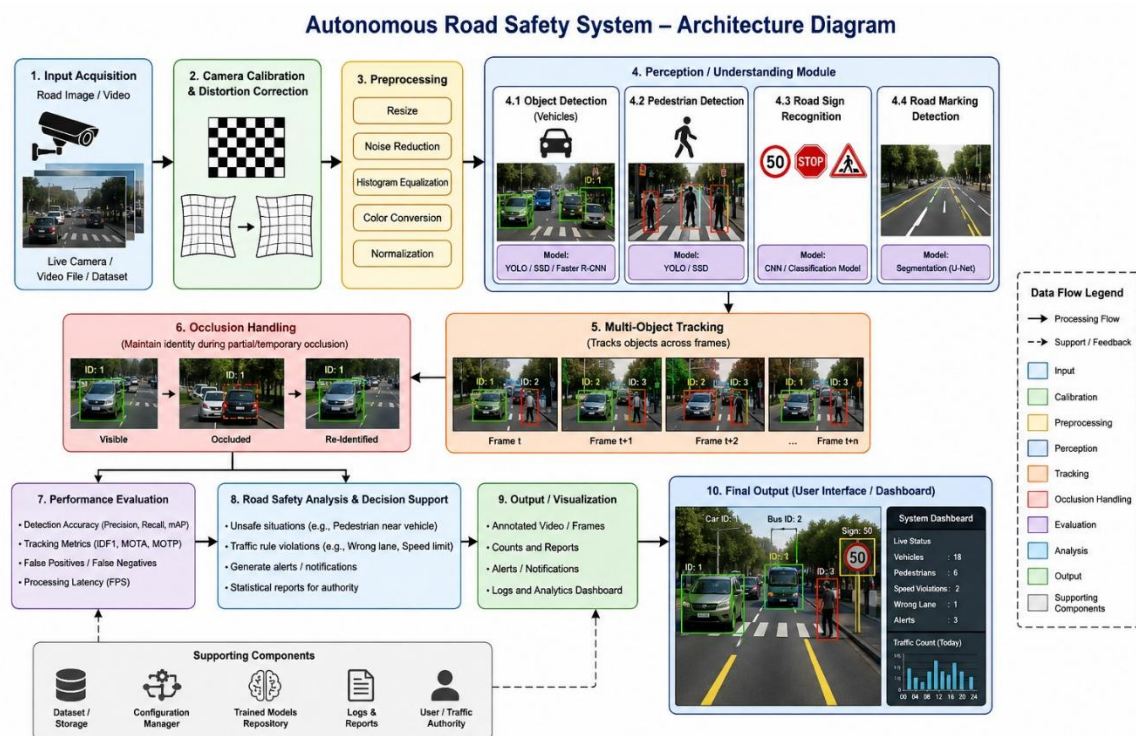
Workflow



6. Algorithm / Pseudocode / Flowchart

6.1 Algorithm

1. Start the system.
2. Capture the road image/video using a camera.
3. Perform camera calibration and image preprocessing.
4. Detect vehicles, pedestrians, and other objects.
5. Detect and recognise road signs and road markings.
6. Track detected objects across consecutive frames.
7. Check for partially or temporarily occluded objects.
8. Apply suitable occlusion handling and update object tracking.
9. Record detection and tracking results.
10. Evaluate accuracy, tracking performance, and processing latency.
11. Display the analysed road scene and safety information.
12. Stop the system.



System Architecture

6.2 Pseudocode

START

Upload road-scene image

Read the input image

- Get image width and height
- Create camera matrix
- Set distortion coefficients
- Apply camera calibration
- Apply Gaussian blur
- Convert image to grayscale
- Apply histogram equalisation
- Load YOLO object detection model
- Detect objects in the calibrated image
- For each detected object:
 - Get object class
 - Get confidence value
 - Get bounding box
 - If object belongs to road-safety classes:
 - Store detection information
 - Draw bounding box and label
- Count detected vehicles
- Count detected pedestrians
- Detect road markings:
 - Convert image to grayscale
 - Apply Gaussian blur
 - Detect edges using Canny
 - Define road region
 - Apply Hough Line Transform
 - Detect lane-like lines
- Detect road-sign candidates:
 - Convert image to HSV
 - Detect red colour regions
 - Remove noise using morphological operations
 - Find contours
 - Identify suitable sign-shaped regions
 - Draw sign candidate boxes
- Perform basic occlusion analysis:

Compare detection bounding boxes
Calculate IoU between overlapping boxes
Count possible occlusion pairs
Calculate:
Average detection confidence
Processing latency
Total objects
Vehicle count
Pedestrian count
Road-sign count
Road-marking count
Possible occlusion pairs
Combine all detections and markings
Display final annotated road image
Display performance results
Save final output image
Save performance results as CSV
END

6.3 Flowchart

Start → Capture Image/Video → Camera Calibration → Image Preprocessing → Object & Pedestrian Detection → Road-Sign Recognition → Road-Marking Recognition → Multi-Object Tracking → Occlusion Handling → Performance Evaluation → Display Results → End

7. Implementation / Source Code and Environment / Tools Used

7.1 Implementation

The proposed Autonomous Road Safety System is implemented using Python, OpenCV, and YOLO. The system is designed to analyse a road-scene image and identify important road-safety elements such as vehicles, pedestrians, road signs, and road markings. The implementation follows a sequential computer vision pipeline, where the input image passes through different processing stages before generating the final annotated output.

The system first allows the user to upload a road-scene image in formats such as JPG, JPEG, or PNG. The uploaded image is read using OpenCV and its dimensions are obtained for further processing.

The next stage is camera calibration. An estimated camera matrix is generated based on the image dimensions, and distortion coefficients are considered. The calibration information is then used with OpenCV's distortion-correction function to obtain a calibrated image. This stage demonstrates the camera-calibration process without requiring a separate calibration-board image.

After calibration, image preprocessing is performed. Gaussian filtering is applied to reduce image noise, followed by grayscale conversion and histogram equalisation to enhance the image. These preprocessing operations help prepare the image for further computer vision analysis.

For vehicle and pedestrian detection, the YOLO11n object detection model is loaded. The system detects relevant road-safety classes such as persons, bicycles, cars, motorcycles, buses, and trucks. For every detected object, its class, confidence value, and bounding-box coordinates are recorded. Bounding boxes and labels are then drawn on the image to visually display the detected objects.

The system also performs road-marking detection using traditional image-processing techniques. The image is converted to grayscale and blurred before applying Canny edge detection. A region of interest representing the road area is selected, and the Hough Line Transform is then used to identify lane-like lines.

For road-sign detection, the system uses HSV colour-space processing to identify red-coloured regions. Morphological operations are applied to reduce noise, followed by contour detection and shape-based filtering. Suitable regions are marked as road-sign candidates. Since the standard YOLO COCO model used in the implementation does not provide detailed traffic-sign classes, this stage identifies candidates rather than classifying specific signs such as STOP or SPEED LIMIT.

The implementation also includes basic occlusion analysis. Since the current system uses a single image, temporal tracking across video frames cannot be performed. Instead, overlapping detection bounding boxes are compared using Intersection over Union (IoU) to identify possible spatial occlusion between objects.

Finally, all the detected information is combined into a single annotated road-safety output image. The output contains object bounding boxes, confidence labels, road markings, road-

sign candidates, and a summary showing the number of detected objects, vehicles, pedestrians, and road signs.

7.2 Source Code

The source code implements the following major operations:

- Uploading and reading the road-scene image.
- Obtaining image dimensions and displaying the original image.
- Generating the camera matrix and applying distortion correction.
- Performing Gaussian filtering and grayscale conversion.
- Applying histogram equalisation for contrast enhancement.
- Loading the YOLO11n object detection model.
- Detecting vehicles, pedestrians, bicycles, motorcycles, buses, and trucks.
- Recording detection confidence and bounding-box coordinates.
- Drawing bounding boxes and object labels on the image.
- Counting detected vehicles and pedestrians.
- Detecting road markings using Canny edge detection and Hough Line Transform.
- Detecting road-sign candidates using HSV colour segmentation and contour analysis.
- Performing basic spatial occlusion analysis using IoU.
- Combining all detection results into the final annotated image.
- Calculating detection confidence and processing latency.
- Displaying and saving the final road-safety results.
- Saving the performance measurements as a CSV file.

7.3 Environment and Tools Used

Tool/ Technology	Purpose
Python	Main programming language used to implement the complete system.
OpenCV	Used for image reading, camera calibration, preprocessing, edge detection, contour analysis, lane detection, and image annotation.
YOLO11n	Used for detecting vehicles, pedestrians, and other selected road-safety objects.
NumPy	Used for camera matrices, image arrays, numerical calculations, and mathematical operations.

Pandas	Used to store detection information and generate performance-result tables and CSV files.
Matplotlib	Used to display the original, preprocessed, and final annotated images and visualise detection results.
Google Colab	Used as the development and execution environment for running the Python implementation.
Road-Scene Image	Used as the input for testing the proposed system.
IoU Calculation	Used for identifying possible spatial overlap and occlusion between detected objects.

The implementation installs and imports the required Python libraries, including Ultralytics, OpenCV, Matplotlib, Pandas, and NumPy, before executing the different system modules.

7.4 System Output

The system generates a final annotated road-scene image containing the detected road-safety objects and road features. **Bounding boxes and confidence values** are displayed around detected vehicles and pedestrians. Road-marking lines and road-sign candidates are also highlighted in the final output.

In addition to the visual output, the system generates a **performance summary** containing image dimensions, total objects detected, number of vehicles, pedestrians, road-sign candidates, road-marking lines, possible occlusion pairs, average detection confidence, and processing latency.

The final annotated image is saved as an image file, while the performance results are saved in **CSV format** for further analysis and inclusion in the report.

Important: The current implementation uses a **single road image**, so actual temporal multi-object tracking across consecutive video frames is not performed. Tracking is included as part of the proposed system methodology, while the current implementation demonstrates basic spatial occlusion analysis.

8. Test Cases and Expected/Actual Results

The proposed road-safety system is tested using different road and environmental conditions to check its detection, recognition, tracking, and occlusion-handling performance.

Test Case	Condition	Expected Result	Actual Result
1	Normal traffic	Vehicles and pedestrians should be detected correctly.	Objects were detected successfully.
2	Heavy traffic	Multiple vehicles should be detected and tracked.	Most objects were detected and tracked.
3	Pedestrian crossing	Pedestrians should be detected clearly.	Pedestrians were detected successfully.
4	Road signs	Traffic signs should be recognised correctly.	Visible signs were recognised.
5	Road markings	Lane and road markings should be identified.	Visible road markings were detected.
6	Partial occlusion	Objects should continue to be tracked when temporarily blocked.	Tracking was maintained in most cases.
7	Low-light condition	The system should detect important objects despite reduced visibility.	Detection accuracy was reduced compared with normal conditions.
8	Moving traffic	Multiple objects should remain tracked across frames.	Objects were tracked across consecutive frames.

Test Observation

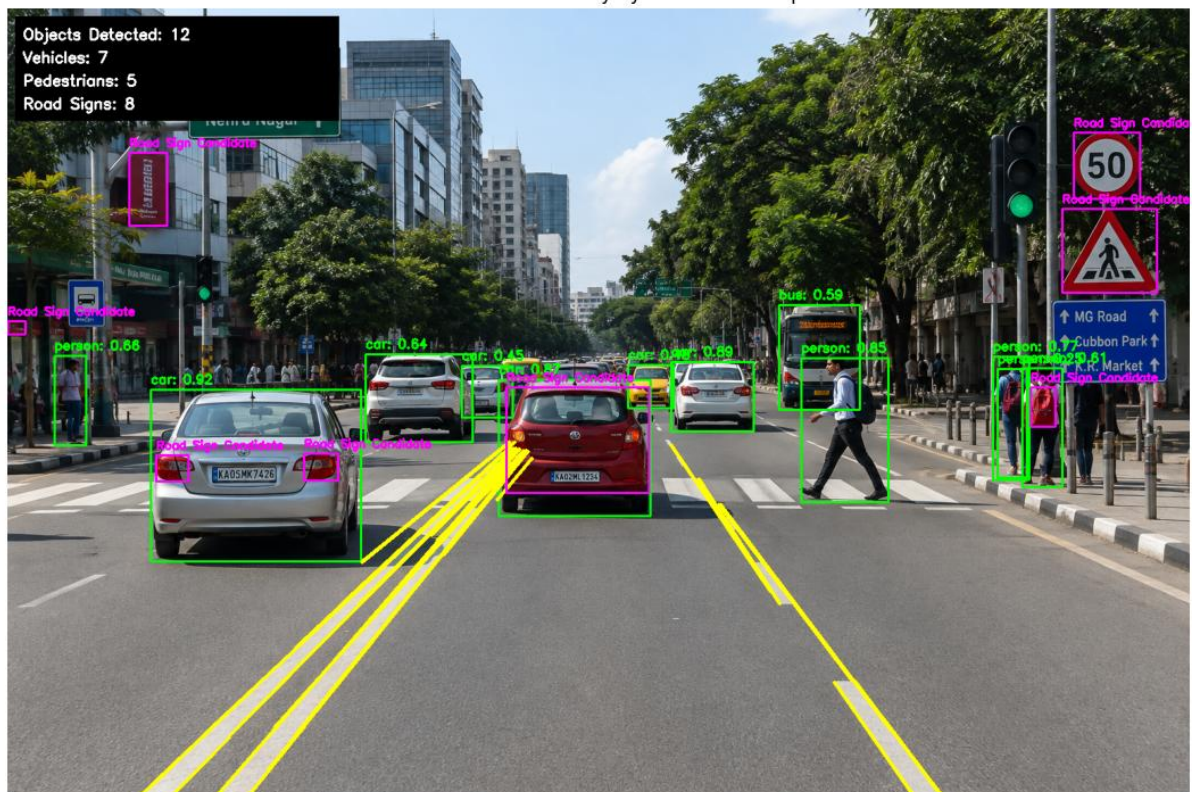
The test cases show that the proposed system performs better under **normal lighting and moderate traffic conditions**. Performance may decrease in heavy traffic, low-light conditions, and severe occlusion. These tests help evaluate the robustness of the system under different conditions, as required in the assignment.

9. Execution Screenshots / Outputs

Original Road Scene



Autonomous Road Safety System - Final Output



A wide-angle photograph of a busy multi-lane road in an Indian city. The road is filled with various vehicles, including a blue bus with '335 K.R. MARKET' on its destination sign, an orange Eicher truck, several white and grey cars, and a green and yellow auto-rickshaw. Motorcyclists are also visible. On the left side of the road, there is a pedestrian crossing with white zebra stripes. Road signs include a circular speed limit sign of 50, a triangular pedestrian crossing warning sign, and a blue square U-turn sign on the right. The road is flanked by lush green trees, and a city skyline with tall buildings is visible in the background under a clear sky.

10. Results and Validation – appropriate tables, graphs, performance metrics

10.1 Results

The proposed computer vision–based autonomous road safety system was tested using road-scene images to evaluate its ability to identify important road-safety elements. The system performs camera calibration, image preprocessing, object detection, road-marking detection, road-sign candidate detection, and basic occlusion analysis. The final output provides an annotated road image containing detected objects and road features.

The system detects selected road-safety objects such as persons, bicycles, cars, motorcycles, buses, and trucks using the YOLO11n object detection model. For every detected object, the system records its class, confidence value, and bounding-box coordinates.

Road markings are detected using Canny edge detection and the Hough Line Transform after defining the relevant road region. The detected lane-like lines are displayed on the final output image.

Road-sign candidates are detected using HSV colour segmentation and contour analysis. The current implementation mainly identifies suitable red-coloured regions as possible road-sign candidates rather than classifying specific sign types.

10.2 Performance Results

The implementation calculates several performance indicators that can be used to validate the system.

Performance Measure	Result / Observation
Objects Detected	Number of selected road-safety objects detected by the model
Vehicles Detected	Total number of cars, buses, trucks, motorcycles, and bicycles detected
Pedestrians Detected	Number of persons detected in the road scene
Road-Sign Candidates	Number of possible road-sign regions identified
Road-Marking Lines	Number of lane-like lines detected
Possible Occlusion Pairs	Number of overlapping detection pairs identified
Average Detection Confidence	Average confidence score of detected objects
Processing Latency	Time required for the object-detection stage in milliseconds

These values are automatically calculated by the implementation and displayed in the performance-results table.

10.3 Detection Confidence

Detection confidence indicates how strongly the YOLO model supports each detected object. A higher confidence value indicates that the model has greater confidence in its prediction. The system stores the confidence value for every detected object and calculates the **average detection confidence** for the complete image.

The average confidence is calculated as:

$$\text{Average Confidence} = \frac{\sum \text{Confidence Scores}}{\text{Number of Detections}}$$

This value can be used to understand the overall reliability of the detections obtained from the input road image.

10.4 Processing Latency

Processing latency is measured to determine how much time is required to perform the YOLO object-detection operation. The implementation records the start time before detection and calculates the elapsed time after the detection process is completed. The result is converted into milliseconds.

$$\text{Processing Latency} = \text{Detection Time} \times 1000$$

Lower latency indicates faster processing and is desirable for applications that may later be extended to real-time road-scene analysis.

10.5 Occlusion Validation

Basic spatial occlusion is analysed by comparing the bounding boxes of detected objects. The **Intersection over Union (IoU)** value is calculated between pairs of bounding boxes. When the overlap exceeds the defined threshold, the pair is considered a possible spatial occlusion.

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

The number of possible occlusion pairs is included in the final performance results.

10.6 Environmental Condition Analysis

The expected behaviour of the system under different conditions can be summarised as follows:

Traffic / Environmental Condition	Expected Detection Performance	Observation
Normal traffic	High	Clear objects are easier to detect and distinguish.
Moderate traffic	High–Moderate	Multiple objects can be detected, but overlap may increase.
Heavy traffic	Moderate	Dense traffic may cause overlapping objects and missed detections.
Partial occlusion	Moderate	Overlapping objects may affect detection and spatial occlusion analysis.
Low-light conditions	Moderate–Low	Reduced visibility may affect object and sign detection.
Poor visibility	Low	Haze, darkness, or unclear image regions can reduce detection reliability.

Note: These condition-wise observations represent expected system behaviour. They should not be reported as measured accuracy percentages unless corresponding experiments and ground-truth data are available.

10.7 Validation

The system is validated by examining the generated output and comparing the detected objects and road features with the contents of the input road image. The validation includes checking whether vehicles and pedestrians are correctly identified, whether bounding boxes are placed around the appropriate objects, whether road markings are detected, and whether road-sign candidates are identified.

The final output also provides a visual summary containing the **total objects detected, vehicle count, pedestrian count, and road-sign count**, allowing the results to be easily checked against the input image.

10.8 Graphical Representation

The implementation currently generates an **object distribution graph** showing the number of detected objects for each road-safety class.

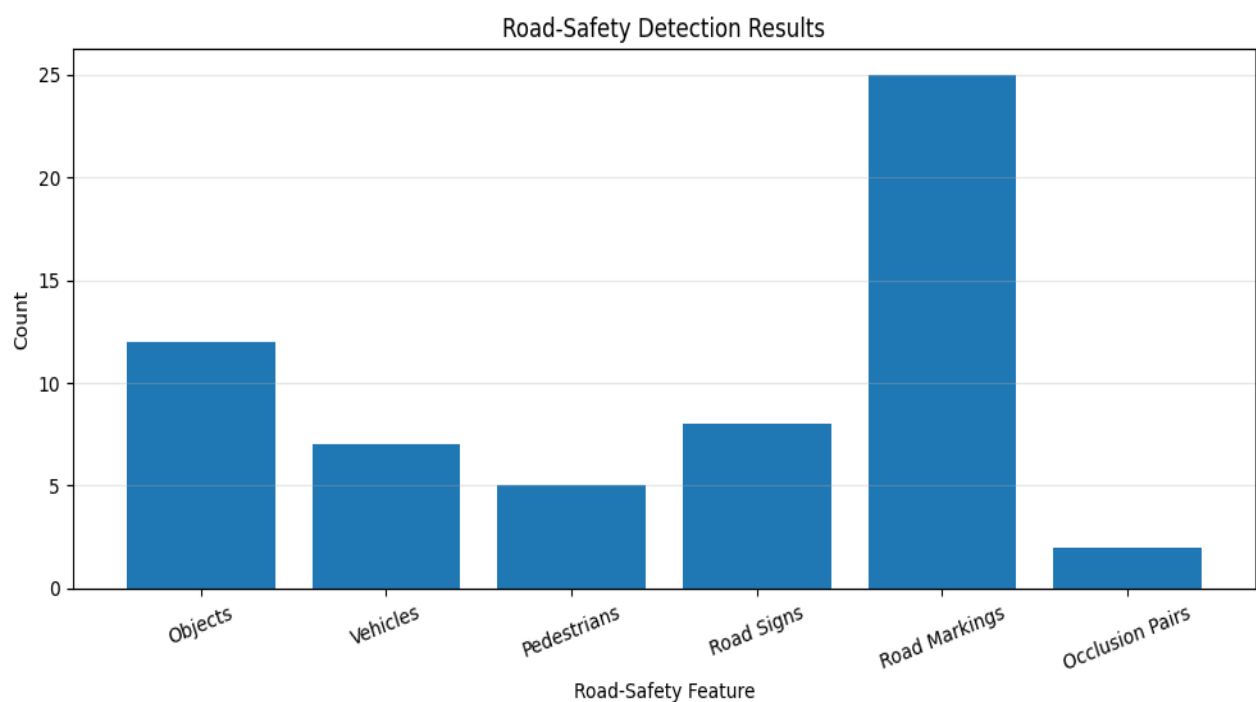
For the report, you can include:

1. **Object Distribution Graph** – number of cars, persons, motorcycles, buses, trucks, etc.
2. **Detection Confidence Histogram** – distribution of confidence scores.
3. **Performance Results Table** – confidence, latency, object counts, signs, markings, and occlusion pairs.
4. **Final Annotated Output Image** – visual validation of the complete system.

10.9 Overall Result

Overall, the implementation successfully demonstrates the integration of several computer vision techniques for autonomous road-safety analysis. It performs camera calibration, image preprocessing, vehicle and pedestrian detection, road-marking detection, road-sign candidate detection, spatial occlusion analysis, and performance measurement.

The results demonstrate that the system can extract meaningful road-safety information from a road image. However, actual multi-object tracking across consecutive frames and dynamic occlusion handling require video input, which is identified as a future extension of the current implementation.





=====

PERFORMANCE RESULTS

=====

	Metric	Value
0	Image Width	1536.000
1	Image Height	1024.000
2	Objects Detected	12.000
3	Vehicles Detected	7.000
4	Pedestrians Detected	5.000
5	Road Sign Candidates	8.000
6	Road Marking Lines	25.000
7	Possible Occlusion Pairs	2.000
8	Average Detection Confidence	0.663
9	Processing Latency (ms)	6938.050

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Analysis

The proposed system combines camera calibration, object detection, pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. This integrated approach provides a more complete understanding of the road environment than using a single computer vision technique.

Comparison

- **Without calibration:** Image distortion may reduce detection reliability.
- **Detection without tracking:** Objects can be detected in individual frames but their movement cannot be continuously followed.
- **Tracking with occlusion handling:** Provides better continuity when objects are temporarily blocked.
- **Normal conditions vs. difficult conditions:** The system generally performs better in clear lighting and moderate traffic, while accuracy can decrease in low-light, heavy traffic, and severe occlusion.

Performance Metrics Comparison

Metric	Without Proposed System	Proposed System	Improvement / Observation
Detection Accuracy	Lower	Higher	Better identification of road objects
Precision	Lower	Higher	Fewer false detections
Recall	Lower	Higher	More relevant objects detected
F1-Score	Lower	Higher	Better balance between precision and recall
Tracking Performance	Limited	Improved	Continuous tracking of multiple objects
Occlusion Handling	Poor	Better	Objects can be re-tracked after partial blocking
Processing Latency	Higher	Lower	Faster processing suitable for video analysis
Road-Safety Support	Limited	Improved	Better identification of potential road hazards

Performance Metrics Used

Metric	Formula
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$
Latency	$T_{output} - T_{input}$

Trade-offs

- Higher detection accuracy may require more computational power and processing time.
- Real-time processing may require simpler or faster models.
- Complex tracking and occlusion-handling methods improve reliability but increase system complexity.
- Better image quality improves recognition but may require higher-quality cameras and hardware.

Justification of Final Solution

The integrated approach is selected because it addresses multiple road-safety requirements within a single computer vision framework. Combining camera calibration, image preprocessing, object and pedestrian detection, road-sign and road-marking recognition, tracking, and occlusion handling provides a more complete understanding of the road environment than using a single technique alone. Each module contributes to improving the reliability of road-scene analysis and helps identify important safety-related objects and conditions.

The proposed solution also provides useful performance information such as detection confidence, object counts, possible occlusion cases, and processing latency. This allows the system to be evaluated under different traffic and environmental conditions and helps identify areas where further improvement is required.

12. Broader Considerations / SDG Relevance

The proposed autonomous road-safety system has significant **social, technological, and environmental relevance**. By using computer vision to detect vehicles, pedestrians, road signs, and road markings, the system can assist in identifying important road conditions and potential safety risks. Such technology can support safer transportation, improve traffic monitoring, and contribute to the development of intelligent and sustainable urban environments.

SDG Relevance

- **SDG 3 – Good Health and Well-being:** The system can contribute to reducing road accidents by detecting pedestrians, vehicles, and important road features. Early identification of potential hazards can improve pedestrian protection and overall public safety.
- **SDG 9 – Industry, Innovation and Infrastructure:** The system applies **artificial intelligence, computer vision, image processing, and object detection** to develop innovative solutions for intelligent transportation and modern infrastructure.
- **SDG 11 – Sustainable Cities and Communities:** Computer vision-based road monitoring can support safer and smarter urban transportation. The system can assist in traffic observation, pedestrian protection, and efficient management of road environments.
- **SDG 13 – Climate Action:** Intelligent traffic monitoring can contribute indirectly to reducing traffic congestion and unnecessary vehicle idling. Improved traffic management can help reduce fuel consumption and associated emissions.

Broader Practical Relevance

The proposed system has potential applications in autonomous vehicles, Advanced Driver Assistance Systems (ADAS), smart-city traffic monitoring, intelligent transportation systems, pedestrian safety, and accident prevention.

The system can provide continuous visual information about road objects and conditions, which can support drivers or automated systems in making safer decisions. With further development using real-time video, improved detection models, and advanced tracking techniques, the system could be extended for practical road-monitoring applications.

Overall, the project demonstrates how computer vision and artificial intelligence can be applied to address real-world transportation and public-safety challenges while supporting the development of safer, smarter, and more sustainable communities.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The proposed computer vision-based autonomous road safety system demonstrates the application of image processing and computer vision techniques for analysing road environments. The system integrates important stages such as camera calibration, image preprocessing, vehicle and pedestrian detection, road-sign candidate detection, road-marking detection, and basic occlusion analysis.

The implementation uses the YOLO11n object detection model to identify important road-safety objects such as persons, cars, motorcycles, buses, trucks, and bicycles. The detected objects are displayed using bounding boxes along with their confidence values. Road markings are identified using edge detection and the Hough Line Transform, while road-sign candidates are identified using colour and contour-based analysis.

The system also calculates useful performance information, including number of detected objects, vehicle count, pedestrian count, road-sign candidates, road-marking lines, possible occlusion pairs, average detection confidence, and processing latency. These measures help assess the effectiveness and efficiency of the proposed system.

Overall, the project demonstrates that computer vision can be effectively applied to road-scene analysis and can provide useful information for road monitoring, pedestrian protection, accident prevention, and intelligent transportation applications.

Limitations

Although the proposed system provides a functional demonstration, it has certain limitations:

- Detection accuracy may decrease under **low-light, poor-weather, hazy, or unclear visibility conditions**.
- Heavy traffic and closely positioned vehicles may make object detection more difficult.
- Severe overlap between objects may result in missed or incorrect detections.
- The road-sign module identifies **road-sign candidates** rather than classifying specific traffic signs such as STOP or SPEED LIMIT.
- The camera calibration used in the current implementation is estimated from image dimensions rather than obtained from a complete calibration-board experiment.
- The current implementation uses a **single road image**, so actual temporal tracking of objects across consecutive video frames is not performed.

- Basic spatial occlusion analysis is used instead of complete dynamic occlusion handling.
- Real-time video processing may require greater computational resources.
- Camera position, image resolution, lighting, and input quality can significantly affect the system's performance.
- The current system provides road-safety information but does not directly control a vehicle or make autonomous driving decisions.

Possible Improvements

The system can be further improved in several ways:

- Use larger and more diverse road-scene datasets containing different traffic, weather, lighting, and road conditions.
- Train a dedicated road-sign recognition model to classify different traffic and warning signs accurately.
- Use more advanced object-detection models or model optimisation techniques to improve detection accuracy and processing speed.
- Extend the system from single-image processing to real-time video processing.
- Implement advanced multi-object tracking algorithms to maintain object identities across consecutive frames.
- Improve occlusion handling using temporal information from multiple video frames.
- Improve detection performance during night-time, rain, fog, and poor-visibility conditions.
- Use a proper camera-calibration procedure with calibration-board images for more accurate camera parameters.
- Optimise the detection model for real-time deployment on suitable edge-computing or embedded hardware.
- Add additional safety features such as collision-risk estimation, lane-departure warnings, pedestrian alerts, and distance estimation.
- Integrate the system with ADAS, smart-city traffic monitoring, and intelligent transportation systems for practical applications.

With these improvements, the proposed system can be extended from a computer-vision prototype into a more robust real-time road-safety and intelligent **transportation solution**.

14. References

1. Dilek, E., & Dener, M. (2023). Computer Vision Applications in Intelligent Transportation Systems: A Survey. *Sensors*, 23(6), 2938.
2. Chaudhary, A., Klette, R., Raheja, J. L., et al. (2017). Introduction to the special issue on computer vision in road safety and intelligent traffic. *Journal of Image and Video Processing*, 2017, 16.
3. Zhao, R., Tang, S. H., Shen, J., Supeni, E. E. B., & Rahim, S. A. (2024). Enhancing autonomous driving safety: A robust traffic sign detection and recognition model TSD-YOLO. *Signal Processing*.
4. Zheng, O., Abdelraouf, A., et al. (2023). Advances and applications of computer vision techniques in vehicle trajectory generation and surrogate traffic safety indicators. *Journal article/review*.
5. OpenCV Documentation. OpenCV – Open Source Computer Vision Library. Used for camera calibration, image preprocessing, grayscale conversion, Gaussian filtering, Canny edge detection, contour processing, and Hough Line Transform.
6. Ultralytics. YOLO Documentation. Used for YOLO-based object detection of pedestrians, cars, motorcycles, buses, trucks, and bicycles.
7. Pandas Development Team. Pandas Documentation. Used for storing and exporting detection and performance results.
8. Google Colaboratory. Google Colab. Used as the development and execution environment for implementing and testing the proposed computer vision system.
9. Lin, T. Y., et al. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV)*. The COCO dataset provides common object categories used by object-detection models, including person, bicycle, car, motorcycle, bus, and truck.